

有限公司SS
022深圳弘源
司SS928 V100
圳弘源泰信息技

深圳弘源泰信息技术有限公司SS
928 V100R001C02SPC022深圳弘源
泰信息技术有限公司SS928 V100
R001C02SPC022深圳弘源泰信息技

有限公司SS
022深圳弘源
司SS928 V100
圳弘源泰信息技

MIPI 使用指南

发布日期 2022-03-10

深圳弘源泰信息技术有限公司SS
928 V100R001C02SPC022深圳弘源
泰信息技术有限公司SS928 V100
R001C02SPC022深圳弘源泰信息技

有限公司SS
022深圳弘源
司SS928 V100
圳弘源泰信息技

深圳弘源泰信息技术有限公司SS
928 V100R001C02SPC022深圳弘源
泰信息技术有限公司SS928 V100
R001C02SPC022深圳弘源泰信息技

前言

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
SS928	V100
SS927	V100

说明

本文以 SS928V100 描述为例，未有特殊说明，SS927V100 与 SS928V100 内容一致。

读者对象

本文档（本指南）主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。

符号	说明
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修订记录

修订记录累积了每次文档更新的说明。最新版本的文档包含以前所有文档版本的更新内容。

文档版本	发布日期	修改说明
00B02	2022-03-10	1.7.1.1 和 1.7.2 小节【调试信息】涉及修改
00B01	2021-05-12	第一次临时版本发布。

目 录

前 言.....	i
1 MIPI 使用指南.....	1
1.1 概述.....	1
1.2 重要概念.....	1
1.3 功能描述.....	3
1.4 API 参考.....	5
1.4.1 OT_MIPI_SET_DEV_ATTR.....	5
1.4.2 OT_MIPI_SET_HS_MODE.....	7
1.4.3 OT_MIPI_SET_PHY_CMVMODE.....	8
1.4.4 OT_MIPI_RESET_SENSOR.....	9
1.4.5 OT_MIPI_UNRESET_SENSOR.....	9
1.4.6 OT_MIPI_RESET_MIPI.....	10
1.4.7 OT_MIPI_UNRESET_MIPI.....	11
1.4.8 OT_MIPI_ENABLE_MIPI_CLOCK.....	12
1.4.9 OT_MIPI_DISABLE_MIPI_CLOCK.....	13
1.4.10 OT_MIPI_ENABLE_SENSOR_CLOCK.....	13
1.4.11 OT_MIPI_DISABLE_SENSOR_CLOCK.....	14
1.4.12 OT_MIPI_SET_EXT_DATA_TYPE.....	15
1.4.13 OT_MIPI_TX_SET_DEV_CFG.....	16
1.4.14 OT_MIPI_TX_SET_CMD.....	16
1.4.15 OT_MIPI_TX_GET_CMD.....	17
1.4.16 OT_MIPI_TX_ENABLE.....	18
1.4.17 OT_MIPI_TX_DISABLE.....	19
1.5 数据类型.....	19
1.5.1 OT_MIPI_IOC_MAGIC.....	21
1.5.2 combo_dev_t.....	22
1.5.3 SNS_MAX_RST_SOURCE_NUM.....	22
1.5.4 SNS_MAX_CLK_SOURCE_NUM.....	23
1.5.5 sns_rst_source_t.....	23

1.5.6 sns_clk_source_t.....	24
1.5.7 MIPI_RX_MAX_DEV_NUM.....	24
1.5.8 COMBO_MAX_LANE_NUM.....	25
1.5.9 MAX_LANE_NUM_PER_LINK.....	25
1.5.10 MIPI_LANE_NUM.....	26
1.5.11 LVDS_LANE_NUM.....	26
1.5.12 WDR_VC_NUM.....	27
1.5.13 SYNC_CODE_NUM.....	27
1.5.14 MAX_EXT_DATA_TYPE_NUM.....	27
1.5.15 lane_divide_mode_t.....	28
1.5.16 input_mode_t.....	28
1.5.17 mipi_data_rate_t.....	29
1.5.18 img_rect_t.....	30
1.5.19 data_type_t.....	30
1.5.20 ext_data_type_t.....	32
1.5.21 mipi_wdr_mode_t.....	33
1.5.22 mipi_dev_attr_t.....	34
1.5.23 lvds_wdr_mode_t.....	35
1.5.24 lvds_sync_mode_t.....	36
1.5.25 lvds_bit_endian_t.....	37
1.5.26 lvds_vsync_type_t.....	37
1.5.27 lvds_vsync_attr_t.....	39
1.5.28 lvds_fid_type_t.....	39
1.5.29 lvds_fid_attr_t.....	40
1.5.30 lvds_dev_attr_t.....	41
1.5.31 phy_cmv_mode_t.....	43
1.5.32 phy_cmv_t.....	43
1.5.33 combo_dev_attr_t.....	44
1.5.34 OT_MIPI_TX_IOC_MAGIC.....	45
1.5.35 LANE_MAX_NUM.....	46
1.5.36 MIPI_TX_SET_DATA_SIZE.....	46
1.5.37 MIPI_TX_GET_DATA_SIZE.....	46
1.5.38 ATTRIBUTE.....	47
1.5.39 out_mode_t.....	47
1.5.40 mipi_tx_work_mode_t.....	48
1.5.41 video_mode_t.....	48
1.5.42 out_format_t.....	49
1.5.43 sync_info_t.....	49
1.5.44 combo_dev_cfg_t.....	50
1.5.45 cmd_info_t.....	52
1.5.46 get_cmd_info_t.....	53

1.6 MIPI TX 模块参数	54
1.6.1 g_smooth.....	54
1.7 Proc 信息.....	55
1.7.1 MIPI_RX Proc 信息.....	55
1.7.2 MIPI_TX Proc 信息.....	62
1.8 FAQ	64
1.8.1 MIPI RX Lane id 如何配置	65
1.8.2 MIPI TX Lane id 如何配置.....	65
1.8.3 MIPI 频率说明	67
1.8.3.1 MIPI Lane 频率与 VI 频率关系	67

插图目录

图 1-1 SOF/EOF/SOL/EOL 同步方式	2
图 1-2 SAV/EAV 同步方式	3
图 1-3 MIPI 数据流	57

表格目录

表 1-1 最大支持 Lane 的个数.....	3
表 1-2 最大对接 sensor 数目.....	4
表 1-3 MIPI Rx Lane 分布模式.....	4
表 1-4 MIPI Rx Lane 管脚关系图.....	4
表 1-5 LVDS 同步方式.....	36
表 1-6 SENSOR 与 MIPI_Rx 管脚关系.....	65
表 1-7 4lane 屏幕与 MIPI_TX 管脚关系.....	66
表 1-8 2lane 屏幕与 MIPI_TX 管脚关系.....	66

1 MIPI 使用指南

1.1 概述

MIPI Rx 通过低电压差分信号接收原始视频数据，将接收到的串行差分信号（serial differential signal）转化为 DC（Digital Camera）时序后传递给下一级模块 VICAP（Video Capture）

MIPI Rx 支持 MIPI D-PHY、LVDS（Low-Voltage Differential Signal）、HiSPi（High-Speed Serial Pixel Interface）等串行视频信号输入，同时兼容 DC 视频接口。

1.2 重要概念

- MIPI
MIPI 的全称是 Mobile Industry Processor Interface(移动行业处理器接口)，本文描述的 MIPI 接口特指物理层使用 D-PHY 传输规范，协议层使用 CSI-2 的通信接口。
- LVDS
LVDS 的全称是 Low Voltage differential Signaling(低压差分信号)，通过同步码区分消隐区和有效数据。
- Lane
用于连接发送端和接收端的一对高速差分线，即可以是时钟 Lane，也可以是数据 Lane。
- 同步码
MIPI 接口使用 CSI-2 里面的短包进行同步，LVDS 使用同步码区分有效数据和消隐区。LVDS 有两种同步方式：

- 使用 SOF/EOF 表示帧起始和结束，使用 SOL/EOL 表示行的起始和结束。同步方式如图 1-1 所示。

图1-1 SOF/EOF/SOL/EOL 同步方式

V.BLK				
H.BLK	SOF	Effective Pixel	EOL	H.BLK
H.BLK	SOL	Effective Pixel		H.BLK
H.BLK		Effective Pixel		H.BLK
H.BLK		Effective Pixel		H.BLK
⋮		⋮		⋮
H.BLK		Effective Pixel		H.BLK
H.BLK		Effective Pixel		H.BLK
H.BLK		Effective Pixel		H.BLK
H.BLK		Effective Pixel		H.BLK
H.BLK		Effective Pixel		H.BLK
H.BLK		Effective Pixel		H.BLK
H.BLK		Effective Pixel		H.BLK
H.BLK		EOF		H.BLK
V.BLK				

- 使用 SAV(invalid) EAV(invalid)表示消隐区的无效数据开始和结束，使用 SAV(valid) EAV(valid)表示有效像素数据的开始和结束。
每个同步码由 4 个字段组成，每个字段的位宽与像素数据位宽保持一致。前 3 个字段为固定基准码字，第 4 个字段由 sensor 厂家确定。
由于不同的 sensor 可能会有不同的同步码，所以需要根据 sensor 配置同步码。同步方式如图 1-2 所示。

图1-2 SAV/EAV 同步方式

H.BLK	SAV (Invalid line)	V.BLK	EAV (Invalid line)	H.BLK
H.BLK		V.BLK		H.BLK
H.BLK		V.BLK		H.BLK
H.BLK	SAV (Valid line)	H.OB / effective pixel	EAV (Valid line)	H.BLK
H.BLK		H.OB / effective pixel		H.BLK
H.BLK		H.OB / effective pixel		H.BLK
⋮		⋮		⋮
H.BLK		H.OB / effective pixel		H.BLK
H.BLK		H.OB / effective pixel		H.BLK
H.BLK		H.OB / effective pixel		H.BLK
H.BLK	SAV (Invalid line)	V.BLK	EAV (Invalid line)	H.BLK
⋮		⋮		⋮
H.BLK		V.BLK		H.BLK
H.BLK		V.BLK		H.BLK

- DOL

DOL 的全称是 Digital Overlap，指 SONY 的 WDR 功能。

1.3 功能描述

MIPI Rx 是一个支持多种差分视频输入接口的采集单元，通过 combo-PHY 接收 MIPI/LVDS/sub-LVDS/HiSpi/DC 接口的数据，通过不同的功能模式配置，MIPI Rx 可以支持多种速度和分辨率的数据传输需求，支持多种外部输入设备。最大支持 Lane 个数如表 1-1 所示。

表1-1 最大支持 Lane 的个数

解决方案	最大支持 lane 数
SS928V100	MIPI Rx 最大支持 8Lane MIPI 输入或 8Lane LVDS 输入。

MIPI Rx 能同时对接多个 sensor，最多对接 sensor 的数目如表 1-2 所示。

表1-2 最大对接 sensor 数目

解决方案	对接 sensor 数目
SS928V100	4

MIPI Rx 最大能同时对接不同数量的 sensor，每个 sensor 需要的 Lane 也不尽相同。因此用户需要确定 MIPI Rx 的 LANE 分布模式。具体的 Lane 分布模式请参见表 1-3。

表1-3 MIPI Rx Lane 分布模式

解决方案	Mode	DEV0	DEV1	DEV2	DEV3
SS928V100	0	L0~L7	N	N	N
	1	L0~L3	N	L4~L7	N
	2	L0~L3	N	L4, L6	L5, L7
	3	L0, L2	L1, L3	L4, L6	L5, L7

MIPI Rx 具体的 Lane 管脚连接请参见表 1-4。

表1-4 MIPI Rx Lane 管脚关系图

解决方案	LANE	MIPI0	MIPI1	MIPI2	MIPI3
SS928V100	Lane0	√	-	-	-
	Lane1	√	√	-	-
	Lane2	√	-	-	-
	Lane3	√	√	-	-
	Lane4	√	-	√	
	Lane5	√	-	√	√
	Lane6	√	-	√	
	Lane7	√	-	√	√

1.4 API 参考

MIPI Rx 提供对接 sensor 时序的功能。提供 ioctl 接口，可用的命令如下：

- `OT_MIPI_SET_DEV_ATTR`：设置 MIPI 设备属性。
- `OT_MIPI_SET_HS_MODE`：设置 MIPI Rx 的 Lane 分布。
- `OT_MIPI_SET_PHY_CMV_MODE`：设置共模电压模式。
- `OT_MIPI_RESET_SENSOR`：复位 sensor。
- `OT_MIPI_UNRESET_SENSOR`：撤销复位 sensor。
- `OT_MIPI_RESET_MIPI`：复位 MIPI Rx。
- `OT_MIPI_UNRESET_MIPI`：撤销复位 MIPI Rx。
- `OT_MIPI_ENABLE_MIPI_CLOCK`：打开 MIPI 设备的时钟。
- `OT_MIPI_DISABLE_MIPI_CLOCK`：关闭 MIPI 设备的时钟。
- `OT_MIPI_ENABLE_SENSOR_CLOCK`：打开 SENSOR 的时钟。
- `OT_MIPI_DISABLE_SENSOR_CLOCK`：关闭 SENSOR 的时钟。
- `OT_MIPI_SET_EXT_DATA_TYPE`：设置 MIPI 扩展 DATA TYPE 的属性。

MIPI Tx 提供对接显示屏、级联的功能。提供 ioctl 接口，可用的命令如下：

- `OT_MIPI_TX_SET_DEV_CFG`：设置 MIPI Tx 设备的属性。
- `OT_MIPI_TX_SET_CMD`：设置发送给 MIPI Tx 设备的命令数据。
- `OT_MIPI_TX_GET_CMD`：用于从外围设备读取信息。
- `OT_MIPI_TX_ENABLE`：使能 MIPI Tx 设备。
- `OT_MIPI_TX_DISABLE`：禁用 MIPI Tx 设备。

1.4.1 OT_MIPI_SET_DEV_ATTR

【描述】

设置 MIPI Rx 设备属性。

【定义】

```
#define OT_MIPI_SET_DEV_ATTR_IOW(OT_MIPI_IOC_MAGIC, 0x01, combo_dev_attr_t)
```

【参数】

`combo_dev_attr_t` 类型的指针。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 errno

【解决方案差异】

无。

【需求】

头文件：ot_mipi_rx.h

【注意】

- 除了配置 `OT_MIPI_SET_DEV_ATTR` 之外，还需要配置以下接口。
- 设置模式：接口为 `OT_MIPI_SET_HS_MODE`。
- 打开 MIPI 时钟：接口为 `OT_MIPI_ENABLE_MIPI_CLOCK`。
- 复位 MIPI：接口为 `OT_MIPI_RESET_MIPI`。
- 打开 SENSOR 的时钟：接口为 `OT_MIPI_ENABLE_SENSOR_CLOCK`。
- 复位 SENSOR：接口为 `OT_MIPI_RESET_SENSOR`。
- 撤销复位 MIPI：接口为 `OT_MIPI_UNRESET_MIPI`。
- 撤销复位 SENSOR：接口为 `OT_MIPI_UNRESET_SENSOR`。
- 推荐的配置流程如下：
 - 设置模式。
 - 打开多路 MIPI 时钟。
 - 复位多路 SENSOR 所对接的 MIPI Rx。
 - 打开多路 SENSOR 所连接的时钟。
 - 复位对接的所有 SENSOR。
 - 配置 MIPI Rx 设备属性。
 - 撤销复位多路 SENSOR 所对接的 MIPI Rx。
 - 撤销复位对接的所有 SENSOR。
- 推荐的退出流程如下：

- a. 复位多路对接的 SENSOR。
- b. 关闭多路 SENSOR 所连接的时钟。
- c. 复位多路 SENSOR 所对接的 MIPI Rx。
- d. 清除多路 SENSOR 所对接的 MIPI Rx 设备的配置。
- e. 关闭多路 MIPI 时钟。
- 操作 SENSOR 复位信号线和时钟信号线会对所连接到该信号线的所有 SENSOR 都产生效果。

【相关数据类型及接口】

- [OT_MIPI_SET_HS_MODE](#)
- [OT_MIPI_RESET_SENSOR](#)
- [OT_MIPI_UNRESET_SENSOR](#)
- [OT_MIPI_RESET_MIPI](#)
- [OT_MIPI_UNRESET_MIPI](#)

1.4.2 OT_MIPI_SET_HS_MODE

【描述】

设置 MIPI Rx 的 Lane 分布模式。

【定义】

```
#define OT_MIPI_SET_HS_MODE_IOW(OT_MIPI_IOC_MAGIC, 0x0b, lane_divide_mode_t)
```

【参数】

[lane_divide_mode_t](#) 类型的指针。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 errno

【解决方案差异】

解决方案	是否支持
SS928V100	支持

【需求】

头文件: ot_mipi_rx.h

【注意】

在接多路 sensor 输入时, 建议在初始时根据硬件连接对整个 lane 分布进行全局的 lane 分布模式的设定, 在之后的多路 sensor 采集过程中不能再调用此接口, 否则可能对其他 sensor 采集有影响。

1.4.3 OT_MIPI_SET_PHY_CMVMODE

【描述】

设置共模电压模式。

【定义】

```
#define OT_MIPI_SET_PHY_CMVMODE_IOW(OT_MIPI_IOC_MAGIC, 0x04, phy_cmvm_t)
```

【参数】

phy_cmvm_t 类型的指针。

【返回值】

返回值	描述
0	成功。
-1	失败, 并设置 errno

【解决方案差异】

解决方案	是否支持
SS928V100	支持

【需求】

头文件: ot_mipi_rx.h

【注意】

无。

1.4.4 OT_MIPI_RESET_SENSOR

【描述】

复位 sensor。

【定义】

```
#define OT_MIPI_RESET_SENSOR_IOW(OT_MIPI_IOC_MAGIC, 0x05, sns_rst_source_t)
```

【参数】

`sns_rst_source_t` SENSOR 复位信号线编号。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 errno。

【解决方案差异】

解决方案	是否支持
SS928V100	支持

【需求】

头文件: ot_mipi_rx.h

【注意】

无。

1.4.5 OT_MIPI_UNRESET_SENSOR

【描述】

撤销复位 sensor。

【定义】

```
#define OT_MIPI_UNRESET_SENSOR_IOW(OT_MIPI_IOC_MAGIC, 0x06, sns_rst_source_t)
```

【参数】

`sns_rst_source_t` SENSOR 复位信号线编号。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 <code>errno</code> 。

【解决方案差异】

解决方案	是否支持
SS928V100	支持

【需求】

头文件: `ot_mipi_rx.h`

【注意】

无。

1.4.6 OT_MIPI_RESET_MIPI

【描述】

复位 MIPI_Rx。

【定义】

```
#define OT_MIPI_RESET_MIPI_IOW(OT_MIPI_IOC_MAGIC, 0x07, combo_dev_t)
```

【参数】

`combo_dev_t` 设备号。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 <code>errno</code>

【解决方案差异】

解决方案类型	是否支持
SS928V100	支持

【需求】

头文件: `ot_mipi_rx.h`

【注意】

无。

1.4.7 OT_MIPI_UNRESET_MIPI

【描述】

撤销复位 MIPI_Rx。

【定义】

```
#define OT_MIPI_UNRESET_MIPI_IOW(OT_MIPI_IOC_MAGIC, 0x08, combo_dev_t)
```

【参数】

`combo_dev_t` 设备号。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 <code>errno</code> 。

【解决方案差异】

解决方案	是否支持
SS928V100	支持

【需求】

头文件: ot_mipi_rx.h

【注意】

无。

1.4.8 OT_MIPI_ENABLE_MIPI_CLOCK

【描述】

打开 MIPI 设备的时钟。

【定义】

```
#define OT_MIPI_ENABLE_MIPI_CLOCK_IOW(OT_MIPI_IOC_MAGIC, 0x0c, combo_dev_t)
```

【参数】

`combo_dev_t` 设备号。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 <code>errno</code> 。

【解决方案差异】

解决方案	是否支持
SS928V100	支持

【需求】

头文件: ot_mipi_rx.h

【注意】

无。

1.4.9 OT_MIPI_DISABLE_MIPI_CLOCK

【描述】

关闭 MIPI 设备的时钟。

【定义】

```
#define OT_MIPI_DISABLE_MIPI_CLOCK_IOW(OT_MIPI_IOC_MAGIC, 0x0d, combo_dev_t)
```

【参数】

`combo_dev_t` 设备号。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 <code>errno</code> 。

【解决方案差异】

解决方案	是否支持
SS928V100	支持

【需求】

头文件: `ot_mipi_rx.h`

【注意】

无。

1.4.10 OT_MIPI_ENABLE_SENSOR_CLOCK

【描述】

打开 SENSOR 的时钟。

【定义】

```
#define OT_MIPI_ENABLE_SENSOR_CLOCK_IOW(OT_MIPI_IOC_MAGIC, 0x10,  
sns_clk_source_t)
```

【参数】

SENSOR 的时钟设备源编号。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 errno

【解决方案差异】

解决方案	是否支持
SS928V100	支持

【需求】

头文件：ot_mipi_rx.h

【注意】

无。

1.4.11 OT_MIPI_DISABLE_SENSOR_CLOCK

【描述】

关闭 SENSOR 的时钟。

【定义】

```
#define OT_MIPI_DISABLE_SENSOR_CLOCK_IOW(OT_MIPI_IOC_MAGIC, 0x11,  
sns_clk_source_t)
```

【参数】

SENSOR 的时钟设备源编号。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 errno。

【解决方案差异】

解决方案	是否支持
SS928V100	支持

【需求】

头文件：ot_mipi_rx.h

【注意】

无。

1.4.12 OT_MIPI_SET_EXT_DATA_TYPE

【描述】

设置 MIPI 扩展 DATA TYPE 的属性。

【定义】

```
#define OT_MIPI_SET_EXT_DATA_TYPE_IOW(OT_MIPI_IOC_MAGIC, 0x12, ext_data_type_t)
```

【参数】

[ext_data_type_t](#) 类型的指针。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 errno。

【需求】

头文件: ot_mipi_rx.h

【注意】

该接口主要用于接收 sensor 内嵌数据, 请注意设备属性中的宽高需要加上内嵌数据的宽高(有效像素宽高+内嵌数据宽高)。

1.4.13 OT_MIPI_TX_SET_DEV_CFG

【描述】

设置 MIPI Tx 设备的属性。

【定义】

```
#define OT_MIPI_TX_SET_DEV_CFG_IOW(OT_MIPI_TX_IOC_MAGIC, 0x01, combo_dev_cfg_t)
```

【参数】

MIPI Tx 设备属性。

【返回值】

返回值	描述
0	成功。
-1	失败, 并设置 errno。

【需求】

头文件: ot_mipi_tx.h

【注意】

- 必须在执行 OT_MIPI_TX_ENABLE 前, 即使能前调用此接口。
- 执行该接口后, MIPI_TX 将默认设置为 LP (Lower Power) 模式, LP 时钟设置为: 关闭。

1.4.14 OT_MIPI_TX_SET_CMD

【描述】

设置发送给 MIPI Tx 设备的命令数据。

【定义】

```
#define OT_MIPI_TX_SET_CMD_IOW(OT_MIPI_TX_IOC_MAGIC, 0x02, cmd_info_t)
```

【参数】

发送给 MIPI Tx 设备的命令信息。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 errno。

【需求】

头文件：ot_mipi_tx.h

【注意】

- 必须在执行 OT_MIPI_TX_ENABLE 前，即使能前调用此接口。
- 必须在执行 OT_MIPI_TX_SET_DEV_CFG 后，即配置设备后调用此接口。
- 此接口执行成功与否依赖硬件或 lane 链路的连通性，连通性异常，则返回失败。

1.4.15 OT_MIPI_TX_GET_CMD

【描述】

用于从外围设备读取信息。

【定义】

```
#define OT_MIPI_TX_GET_CMD_IOWR(OT_MIPI_TX_IOC_MAGIC, 0x04, get_cmd_info_t)
```

【参数】

详见 get_cmd_info_t 结构体说明。

【返回值】

返回值	描述
0	成功。

返回值	描述
-1	失败，并设置 errno。

【需求】

头文件：ot_mipi_tx.h

【注意】

- 在执行 OT_MIPI_TX_ENABLE 后，调用此接口，存在概率读失败的情况。
- 必须在执行 OT_MIPI_TX_SET_DEV_CFG 后，即配置设备后调用此接口。
- 此接口执行成功与否依赖硬件或 lane 链路的连通性，连通性异常，则返回失败。

1.4.16 OT_MIPI_TX_ENABLE

【描述】

使能 MIPI Tx 设备。

【定义】

```
#define OT_MIPI_TX_ENABLE_IO(OT_MIPI_TX_IOC_MAGIC, 0x03)
```

【参数】

无。

【返回值】

返回值	描述
0	成功。
-1	失败，并设置 errno。

【需求】

头文件：ot_mipi_tx.h

【注意】

- 使能前，必须调用 OT_MIPI_TX_SET_DEV_CFG 对设备进行配置。

- 此接口调用后 MIPI_TX 将工作于 HS 模式 (High Speed), LP 时钟设置为: 打开。

1.4.17 OT_MIPI_TX_DISABLE

【描述】

禁用 MIPI Tx 设备。

【定义】

```
#define OT_MIPI_TX_DISABLE_IO(OT_MIPI_TX_IOC_MAGIC, 0x05)
```

【参数】

无。

【返回值】

返回值	描述
0	成功。
-1	失败, 并设置 errno。

【需求】

头文件: ot_mipi_tx.h

【注意】

- 此接口调用后 MIPI_TX 将工作于 LP 模式 (Lower Power), LP 时钟设置为: 关闭。
- 设备禁用后需要使用 OT_MIPI_TX_SET_DEV_CFG 重新设置设备属性, 才可使用设备。

1.5 数据类型

MIPI Rx 相关数据类型定义如下:

- OT_MIPI_IOC_MAGIC: MIPI Rx ioctl 命令的幻数。
- combo_dev_t: MIPI Rx 设备类型。

- `SNS_MAX_RST_SOURCE_NUM`: SENSOR 的复位信号线个数。
- `SNS_MAX_CLK_SOURCE_NUM`: SENSOR 的时钟信号线个数。
- `sns_rst_source_t`: SENSOR 的复位信号线编号, 软件上称为 SENSOR 的复位源。
- `sns_clk_source_t`: SENSOR 的时钟信号线编号, 软件上称为 SENSOR 的时钟源。
- `MIPI_RX_MAX_DEV_NUM`: MIPI Rx 支持的设备数。
- `COMBO_MAX_LANE_NUM`: 设备最大支持的 Lane 数量。
- `MAX_LANE_NUM_PER_LINK`: MIPI Rx 一个 link 的 Lane 数。
- `MIPI_LANE_NUM`: MIPI Rx 的 MIPI 设备支持的最大 Lane 数。
- `LVDS_LANE_NUM`: LVDS/HiSPi 接口支持的 Lane 数量。
- `WDR_VC_NUM`: 定义最多支持的 Virtual Chnnael 数量。
- `SYNC_CODE_NUM`: 定义 LVDS 每个 Virtual Channel 的同步码数量。
- `MAX_EXT_DATA_TYPE_NUM`: 定义扩展 DATE TYPE 的数量。
- `lane_divide_mode_t`: MIPI Rx 的 Lane 分布。
- `input_mode_t`: MIPI Rx 输入接口类型。
- `mipi_data_rate_t`: MIPI Rx 输入速率。
- `img_rect_t`: crop 属性。
- `data_type_t`: 传输的数据类型。
- `ext_data_type_t`: MIPI 扩展 data type 属性。
- `mipi_wdr_mode_t`: MIPI WDR 模式。
- `mipi_dev_attr_t`: MIPI 设备属性。
- `lvds_wdr_mode_t`: LVDS WDR 模式。
- `lvds_sync_mode_t`: LVDS 同步方式。
- `lvds_bit_endian_t`: 比特位大小端模式。
- `lvds_vsync_type_t`: LVDS vsync 类型。
- `lvds_vsync_attr_t`: LVDS vsync 参数。
- `lvds_fid_type_t`: Frame identification Id 类型。
- `lvds_fid_attr_t`: Frame indentification Id 配置信息。
- `lvds_dev_attr_t`: LVDS/SubLVDS/HiSPi 设备属性。
- `phy_cmv_mode_t`: PHY 共模电压模式。

- `phy_cmv_t`: PHY 共模电压配置信息。
- `combo_dev_attr_t`: combo 设备属性。
- `OT_MIPI_TX_IOC_MAGIC`: MIPI Tx ioctl 命令的幻数。
- `LANE_MAX_NUM`: 定义 MIPI Tx 支持的最大 Lane 数。
- `MIPI_TX_SET_DATA_SIZE`: 定义 MIPI TX 长指令支持的最大数据长度。
- `MIPI_TX_GET_DATA_SIZE`: 定义 MIPI TX 读指令支持的最大数据长度。
- `ATTRIBUTE`: 定义编译时对齐字节数。
- `out_mode_t`: MIPI Tx 输出或外设操作模式。
- `mipl_tx_work_mode_t`: MIPI Tx 工作模式。
- `video_mode_t`: MIPI Tx 视频模式或视频格式, 或包序列格式。
- `out_format_t`: MIPI Tx 输出数据格式。
- `sync_info_t`: MIPI Tx 设备同步信息。
- `combo_dev_cfg_t`: MIPI Tx 设备属性。
- `cmd_info_t`: 发送给 MIPI Tx 设备的命令信息。
- `get_cmd_info_t`: 发送给 MIPI Tx 设备的命令信息。

1.5.1 OT_MIPI_IOC_MAGIC

【说明】

MIPI Rx ioctl 命令的幻数。

【定义】

```
#define OT_MIPI_IOC_MAGIC 'm'
```

【成员】

无

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

无

1.5.2 combo_dev_t

【说明】

MIPI Rx 设备类型。

【定义】

```
typedef unsigned int combo_dev_t;
```

【解决方案差异】

解决方案	MIPI 设备范围
SS928V100	[0, MIPI_RX_MAX_DEV_NUM)

【注意事项】

无。

【相关数据类型及接口】

- [combo_dev_attr_t](#)
- [OT_MIPI_SET_DEV_ATTR](#)
- [OT_MIPI_RESET_MIPI](#)
- [OT_MIPI_UNRESET_MIPI](#)

1.5.3 SNS_MAX_RST_SOURCE_NUM

【说明】

SENSOR 的复位信号线个数。

【定义】

```
#define SNS_MAX_RST_SOURCE_NUM 4
```

【解决方案差异】

解决方案	SENSOR 复位信号线数目
SS928V100	4

【注意事项】

无。

【相关数据类型及接口】

无

1.5.4 SNS_MAX_CLK_SOURCE_NUM

【说明】

SENSOR 的时钟信号线个数。

【定义】

#define SNS_MAX_CLK_SOURCE_NUM 4

【解决方案差异】

解决方案	SENSOR 时钟信号线数目
SS928V100	4

【注意事项】

无

【相关数据类型及接口】

无

1.5.5 sns_rst_source_t

【说明】

SENSOR 的复位信号线编号，软件上称为 SENSOR 的复位源。

【定义】

typedef unsigned int sns_rst_source_t;

【解决方案差异】

解决方案	SENSOR 复位设备范围
SS928V100	[0, SNS_MAX_RST_SOURCE_NUM)

【注意事项】

每条 SENSOR 复位信号线可以接两个 SENSOR，用户需要根据板子的连线确认 SENSOR 复位信号线编号。不同的芯片的 SENSOR 复位信号线数目请参考取值范围。

【相关数据类型及接口】

无。

1.5.6 sns_clk_source_t

【说明】

SENSOR 的时钟信号线编号，软件上称为 SENSOR 的时钟源。

【定义】

```
typedef unsigned int sns_clk_source_t;
```

【解决方案差异】

解决方案	SENSOR 时钟设备范围
SS928V100	[0, SNS_MAX_CLK_SOURCE_NUM)

【注意事项】

每条 SENSOR 时钟信号线可以接两个 SENSOR，用户需要根据板子的连线确认 SENSOR 时钟信号线编号。不同的芯片的 SENSOR 复位信号线数目请参考取值范围。

【相关数据类型及接口】

- OT_MIPI_ENABLE_SENSOR_CLOCK
- OT_MIPI_DISABLE_SENSOR_CLOCK

1.5.7 MIPI_RX_MAX_DEV_NUM

【说明】

MIPI Rx 支持的设备数。

【定义】

```
#define MIPI_RX_MAX_DEV_NUM 4
```

【解决方案差异】

解决方案	MIPI Rx 支持的设备数
SS928V100	4

【注意事项】

SS928V100 支持多个 MIPI Rx 设备同时使用。

【相关数据类型及接口】

无

1.5.8 COMBO_MAX_LANE_NUM

【说明】

MIPI Rx 的 Lane 总数目。

【定义】

```
#define COMBO_MAX_LANE_NUM 8
```

【解决方案差异】

无

【注意事项】

无

【相关数据类型及接口】

无

1.5.9 MAX_LANE_NUM_PER_LINK

【说明】

MIPI Rx 一个 link 的 Lane 数。

【定义】

```
#define MAX_LANE_NUM_PER_LINK 2
```

【解决方案差异】

无

【注意事项】

这里的 link 是软件概念，软件上把一个逻辑的 link 拆分成了 2 个软件的 link。

【相关数据类型及接口】

无

1.5.10 MIPI_LANE_NUM

【说明】

MIPI Rx 的 MIPI 设备支持的最大 Lane 数。

【定义】

```
#define MIPI_LANE_NUM 8
```

【解决方案差异】

无

【注意事项】

无

【相关数据类型及接口】

无

1.5.11 LVDS_LANE_NUM

【说明】

MIPI Rx 的 LVDS 设备支持的最大 Lane 数。

【定义】

```
#define LVDS_LANE_NUM 8
```

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.12 WDR_VC_NUM

【说明】

定义最多支持的 Virtual Channel 数量。

【定义】

```
#define WDR_VC_NUM 4
```

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.13 SYNC_CODE_NUM

【说明】

定义 LVDS 每个 Virtual Channel 的同步码数量

【定义】

```
#define SYNC_CODE_NUM 4
```

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.14 MAX_EXT_DATA_TYPE_NUM

【说明】

定义扩展 DATE TYPE 的数量。

【定义】

```
#define MAX_EXT_DATA_TYPE_NUM 3
```

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

[ext_data_type_t](#)

1.5.15 lane_divide_mode_t

【说明】

MIPI Rx 的 LANE 分布。

【定义】

```
typedef enum {  
    LANE_DIVIDE_MODE_0 = 0, /* 8lane */  
    LANE_DIVIDE_MODE_1 = 1, /* 4lane + 4lane */  
    LANE_DIVIDE_MODE_2 = 2, /* 4lane + 2lane + 2lane */  
    LANE_DIVIDE_MODE_3 = 3, /* 2lane + 2lane + 2lane + 2lane */  
    LANE_DIVIDE_MODE_BUTT  
} lane_divide_mode_t;
```

【解决方案差异】

解决方案	MIPI Rx 的 LANE 分布
SS928V100	[LANE_DIVIDE_MODE_0, LANE_DIVIDE_MODE_3]

【注意事项】

只有 MIPI 需要设置 LANE 的分布。

【相关数据类型及接口】

[OT_MIPI_SET_HS_MODE](#)

1.5.16 input_mode_t

【说明】

MIPI Rx 输入接口类型

【定义】

```
typedef enum {
    INPUT_MODE_MIPI = 0x0,          /* mipi */
    INPUT_MODE_SUBLVDS = 0x1,       /* SUB_LVDS */
    INPUT_MODE_LVDS = 0x2,          /* LVDS */
    INPUT_MODE_HISPI = 0x3,         /* HISPI */
    INPUT_MODE_BUTT
} input_mode_t;
```

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.17 mipi_data_rate_t

【说明】

MIPI Rx 输入速率。

【定义】

```
typedef enum
{
    MIPI_DATA_RATE_X1 = 0,          /* output 1 pixel per clock */
    MIPI_DATA_RATE_X2 = 1,          /* output 2 pixel per clock */
    MIPI_DATA_RATE_BUTT
} mipi_data_rate_t;
```

【解决方案差异】

解决方案	是否支持 MIPI_DATA_RATE_X2
SS928V100	MIPI 的所有设备都支持。

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.18 img_rect_t

【说明】

Mipi crop 属性。

【定义】

```
typedef struct
{
    int x;
    int y;
    unsigned int width;
    unsigned int height;
} img_rect_t;
```

【成员】

成员名称	描述
x	Crop 起始位置 x。
y	Crop 起始位置 y。
width	Crop 宽度，单位：像素。
height	Crop 高度单位：像素。

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.19 data_type_t

【说明】

传输的数据类型。

【定义】

```
typedef enum
```

```
{
```

```
    DATA_TYPE_RAW_8BIT = 0,
```

```
    DATA_TYPE_RAW_10BIT,
```

```
    DATA_TYPE_RAW_12BIT,
```

```
    DATA_TYPE_RAW_14BIT,
```

```
    DATA_TYPE_RAW_16BIT,
```

```
    DATA_TYPE_YUV420_8BIT_NORMAL,
```

```
    DATA_TYPE_YUV420_8BIT_LEGACY,
```

```
    DATA_TYPE_YUV422_8BIT,
```

```
    DATA_TYPE_YUV422_PACKED,
```

```
    DATA_TYPE_BUTT
```

```
} data_type_t;
```

【成员】

成员名称	描述
DATA_TYPE_RAW_8BIT	8BIT 的 RAW 数据。
DATA_TYPE_RAW_10BIT	10BIT 的 RAW 数据。
DATA_TYPE_RAW_12BIT	12BIT 的 RAW 数据。
DATA_TYPE_RAW_14BIT	14BIT 的 RAW 数据。
DATA_TYPE_RAW_16BIT	16BIT 的 RAW 数据。
DATA_TYPE_YUV420_8BIT_NORMAL	8BIT 的 YUV420 数据，NORMAL 模式。
DATA_TYPE_YUV420_8BIT_LEGACY	8BIT 的 YUV420 数据，LEGACY 模式。
DATA_TYPE_YUV422_8BIT	8BIT 的 YUV422 数据。
DATA_TYPE_YUV422_PACKED	YUV422 数据在 MIPI 按 16bit raw 打包接收。

【解决方案差异】

解决方案	支持的数据类型
SS928V100	全部支持

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.20 ext_data_type_t

【说明】

MIPI 扩展 DATE TYPE 属性。

【定义】

```
typedef struct {
    combo_dev_t devno;
    unsigned int num;
    unsigned int ext_data_bit_width[MAX_EXT_DATA_TYPE_NUM];
    unsigned int ext_data_type[MAX_EXT_DATA_TYPE_NUM];
} ext_data_type_t;
```

【成员】

成员名称	描述
devno	MIPI Rx 设备号
num	支持的扩展 DATE TYPE 数量
ext_data_bit_width	对应扩展 DATE TYPE 输入数据的位宽
ext_data_type	对应扩展 DATE TYPE 的值

【解决方案差异】

无

【注意事项】

无

【相关数据类型及接口】

[OT_MIPI_SET_EXT_DATA_TYPE](#)

1.5.21 mipi_wdr_mode_t

【说明】

MIPI WDR 模式。

【定义】

```
typedef enum
{
    OT_MIPI_WDR_MODE_NONE = 0x0,
    OT_MIPI_WDR_MODE_VC   = 0x1,   /* Virtual Channel */
    OT_MIPI_WDR_MODE_DT   = 0x2,   /* Data Type */
    OT_MIPI_WDR_MODE_DOL  = 0x3,   /* DOL Mode */
    OT_MIPI_WDR_MODE_BUTT
} mipi_wdr_mode_t;
```

【成员】

成员名称	描述
OT_MIPI_WDR_MODE_NONE	线性模式
OT_MIPI_WDR_MODE_VC	使用 Packet header 中的 Virtual Channel 区分长短曝光帧
OT_MIPI_WDR_MODE_DT	使用 Packet header 中的自定义 Data type 区分长短曝光帧
OT_MIPI_WDR_MODE_DOL	表示 DOL 模式 WDR，使用 Packet header 之后的一个 pixel 识别长短曝光帧

【解决方案差异】

无

【注意事项】

无

【相关数据类型及接口】

无

1.5.22 mipi_dev_attr_t

【说明】

mipi 设备属性。

【定义】

```
typedef struct
{
    data_type_t        input_data_type;
    mipi_wdr_mode_t    wdr_mode;
    short              lane_id[MIPI_LANE_NUM];
    union
    {
        short data_type[WDR_VC_NUM];
    };
} mipi_dev_attr_t;
```

【成员】

成员名称	描述
input_data_type	传输的数据类型。
lane_id	发送端(sensor)和接收端(MIPI Rx) Lane 的对应关系 未使用的 Lane 设置为-1。
wdr_mode	MIPI WDR 模式
data_type	当 wdr_mode 为 OT_MIPI_WDR_MODE_DT 时，需要设置 data_type，表示不同曝光长度数据对应的 Data Type。

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

- [data_type_t](#)
- [mipi_wdr_mode_t](#)
- [OT_MIPI_SET_DEV_ATTR](#)

1.5.23 lvds_wdr_mode_t

【说明】

LVDS WDR 模式。

【定义】

```
typedef enum
{
    OT_WDR_MODE_NONE = 0x0,
    OT_WDR_MODE_2F = 0x1,
    OT_WDR_MODE_3F = 0x2,
    OT_WDR_MODE_4F = 0x3,
    OT_WDR_MODE_DOL_2F = 0x4,
    OT_WDR_MODE_DOL_3F = 0x5,
    OT_WDR_MODE_DOL_4F = 0x6,
    OT_WDR_MODE_BUTT
} lvds_wdr_mode_t;
```

【成员】

成员名称	描述
OT_WDR_MODE_NONE	线性模式
OT_WDR_MODE_2F	2 合一 WDR
OT_WDR_MODE_3F	3 合一 WDR
OT_WDR_MODE_4F	4 合一 WDR
OT_WDR_MODE_DOL_2F	DOL 模式 2 合一 WDR
OT_WDR_MODE_DOL_3F	DOL 模式 3 合一 WDR
OT_WDR_MODE_DOL_4F	DOL 模式 4 合一 WDR

【解决方案差异】

解决方案	WDR 模式
SS928V100	都支持

【注意事项】

- DOL WDR 模式需要配置为 OT_WDR_MODE_DOL_2F/ OT_WDR_MODE_DOL_3F/ OT_WDR_MODE_DOL_4F。
- Built-in WDR 模式和帧合成 WDR 模式都需要配置为 OT_WDR_MODE_NONE。

【相关数据类型及接口】

无。

1.5.24 lvds_sync_mode_t

【说明】

LVDS 同步方式。

【定义】

```
typedef enum
{
    LVDS_SYNC_MODE_SOF = 0,          /* sensor SOL, EOL, SOF, EOF */
    LVDS_SYNC_MODE_SAV,              /* SAV, EAV */
    LVDS_SYNC_MODE_BUTT
} lvds_sync_mode_t;
```

表1-5 LVDS 同步方式

sync_mode	同步方式
LVDS_SYNC_MODE_SOF	SOF、EOF、SOL、EOL 请参考图 1-1。
LVDS_SYNC_MODE_SAV	invalid SAV、invalid EAV、valid SAV、valid EAV 请参考图 1-2。

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.25 lvds_bit_endian_t

【说明】

比特位大小端模式。

【定义】

```
typedef enum
{
    LVDS_ENDIAN_LITTLE = 0x0,
    LVDS_ENDIAN_BIG    = 0x1,
    LVDS_ENDIAN_BUTT
} lvds_bit_endian_t;
```

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.26 lvds_vsync_type_t

【说明】

LVDS vsync 类型。

【定义】

```
typedef enum
{
    LVDS_VSYNC_NORMAL    = 0x00,
    LVDS_VSYNC_SHARE     = 0x01,
    LVDS_VSYNC_HCONNECT = 0x02,
    LVDS_VSYNC_BUTT
}
```

```
} lvds_vsync_type_t;
```

【成员】

成员名称	描述
LVDS_VSYNC_NORMAL	长短曝光帧有独立的 SOF-EOF、SOL-EOL 或者 invalid SAV-invalid EAV, valid SAV-valid EAV。
LVDS_VSYNC_SHARE	长短曝光帧共用一对 SOF-EOF 标识, 短曝光的起始几行用固定值填充。
LVDS_VSYNC_HCONNECT	长短曝光帧共用一对 SAV-EAV 标识, 长短曝光帧之间是固定周期的消隐。

• LVDS_VSYNC_SHARE 同步方式:

SOF	Long Exposure	EOL	Horizontal Blanking	SOL	Padding	EOL	Horizontal Blanking
SOL					Short Exposure		
	Padding					EOF	
SOV	V.BLK	EOV	-	SOV	V.BLK	EOV	-

• LVDS_VSYNC_HCONNECT 同步方式:

SAV	Long Exposure frame	Horizontal Blanking (fix period)	V.BLK	Horizontal Blanking (fix period)	V.BLK	EAV	Horizontal Blanking
			Short Exposure Frame1		Short Exposure Frame2		
	V.BLK		V.BLK				
	V.BLK						-

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

[lvds_vsync_attr_t](#)

1.5.27 lvds_vsync_attr_t

【说明】

LVDS vsync 参数

【定义】

```
typedef struct
{
    lvds_vsync_type_t sync_type;
    unsigned short hblank1;
    unsigned short hblank2;
} lvds_vsync_attr_t;
```

【解决方案差异】

无。

【注意事项】

当 sync_type 为 [LVDS_VSYNC_HCONNECT](#) 时，需要配置 hblank1 和 hblank2，表示 Hconnect 的消隐区长度。

【相关数据类型及接口】

[lvds_vsync_type_t](#)

1.5.28 lvds_fid_type_t

【说明】

Frame identification Id 类型

【定义】

```
typedef enum
{
    LVDS_FID_NONE      = 0x00,
    LVDS_FID_IN_SAV    = 0x01,    /* frame identification id in SAV 4th */
    LVDS_FID_IN_DATA    = 0x02,    /* frame identification id in first data */
    LVDS_FID_BUTT
}
```

```
} lvds_fid_type_t;
```

【成员】

成员名称	描述
LVDS_FID_NONE	不使用 frame identification id。
LVDS_FID_IN_SAV	FID 插入在 SAV 第 4 个字段中，DOL 4 个字段的同步码需要将 fid_type 配置为 LVDS_FID_IN_SAV。
LVDS_FID_IN_DATA	FID 作为 Frame information column 插入在同步码之后的第一个像素之前，DOL 5 个字段的同步码需要将 fid_type 配置为 LVDS_FID_IN_DATA。

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.29 lvds_fid_attr_t

【说明】

Frame identification Id 配置信息。

【定义】

```
typedef struct
{
    lvds_fid_type_t fid_type;
    unsigned char output_fil;
} lvds_fid_attr_t;
```

【成员】

成员名称	描述
fid_type	LVDS DOL 模式下的 Frame identification Id 类型

成员名称	描述
output_fil	DOL 模式中的 Frame information line 紧接在 V-Blanking 之后输出，Frame ID 是 Frame information line 中的第一个像素值。 Frame information line 中并不包含有效的视频数据： - 如果 output_fil 设置为 1，Frame information line 会输出到后端设备。 - 如果 output_fil 设置为 0，MIPI Rx 将丢弃这一行数据。

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

[lvds_fid_type_t](#)

1.5.30 lvds_dev_attr_t

【说明】

LVDS/SubLVDS/HiSPi 设备属性。

【定义】

```
typedef struct
{
    data_type_t        input_data_type;
    lvds_wdr_mode_t    wdr_mode;
    lvds_sync_mode_t   sync_mode;
    lvds_vsync_attr_t  vsync_attr;
    lvds_fid_attr_t     fid_attr;
    lvds_bit_endian_t  data_endian;
    lvds_bit_endian_t  sync_code_endian;
    short              lane_id[LVDS_LANE_NUM];
    unsigned short      sync_code[LVDS_LANE_NUM][WDR_VC_NUM][SYNC_CODE_NUM];
} lvds_dev_attr_t;
```

【成员】

成员名称	描述
input_data_type	传输的数据类型。
wdr_mode	WDR 模式。
sync_mode	LVDS 同步模式。
vsync_attr	vsync 类型, 当 wdr_mod 为 DOL 模式并且 sync_mode 为 LVDS_SYNC_MODE_SAV 时, 需要配置 vsync 的类型。
fid_attr	frame identification 类型, 当 wdr_mode 为 DOL 模式, 并且 sync_mode 为 LVDS_SYNC_MODE_SAV 时, 需要配置。
data_endian	数据大小端模式。
sync_code_endian	同步码大小端模式。
lane_id	发送端(sensor)和接收端(MIPI Rx) lane 的对应关系 未使用的 lane 设置为-1 lane id 的配置方式请参考“ MIPI RX Lane id 如何配置 ”。
sync_code	每个 Virtual Channel 有 4 个同步码, 根据同步模式不同, 分别表示 SOF/EOF/ SOL/EOL 的同步码或者 invalid SAV/invalid EAV/ valid SAV/valid EAV 的同步码。

【解决方案差异】

无。

【注意事项】

使用该结构体, LVDS 的 LANE 同步码同步到达。

【相关数据类型及接口】

- [lvds_wdr_mode_t](#)
- [lvds_sync_mode_t](#)
- [data_type_t](#)
- [lvds_bit_endian_t](#)
- [lvds_vsync_type_t](#)
- [lvds_fid_type_t](#)
- [OT_MIPI_SET_DEV_ATTR](#)

1.5.31 phy_cmv_mode_t

【说明】

PHY 共模电压模式。

【定义】

```
typedef enum
{
    PHY_CMV_GE1200MV    = 0x00,
    PHY_CMV_LT1200MV    = 0x01,
    PHY_CMV_BUTT
} phy_cmv_mode_t;
```

【成员】

成员名称	描述
PHY_CMV_GE1200MV	PHY 共模电压大于等于 1200mv。
PHY_CMV_LT1200MV	PHY 共模电压小于 1200 mv。

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.32 phy_cmv_t

【说明】

PHY 共模电压配置信息。

【定义】

```
typedef struct
{
    combo_dev_t      devno;
    phy_cmv_mode_t   cmv_mode;
```

```
} phy_cmv_t;
```

【成员】

成员名称	描述
devno	MIPI Rx 设备号。
cmv_mode	PHY 功能电压模式。

【解决方案差异】

无。

【注意事项】

无。

【相关数据类型及接口】

- [phy_cmv_mode_t](#)
- [OT_MIPI_SET_PHY_CMVMODE](#)

1.5.33 combo_dev_attr_t

【说明】

combo 设备属性，由于 MIPI Rx 能够对接 CSI-2、LVDS、HiSpi 等时序，所以将 MIPI Rx 称为 combo 设备。

【定义】

```
typedef struct
{
    combo_dev_t      devno;
    input_mode_t      input_mode;
    mipi_data_rate_t  data_rate;
    img_rect_t        img_rect;
    union
    {
        mipi_dev_attr_t  mipi_attr;
        lvds_dev_attr_t  lvds_attr;
    };
} combo_dev_attr_t;
```

【成员】

成员名称	描述
devno	MIPI Rx 设备号
input_mode	输入接口类型。
data_rate	接口传输速率。
img_rect	图像 crop 区域。
mipi_attr	如果 input_mode 配置为 INPUT_MODE_MIPI, 则必须配置 mipi_attr。
lvds_attr	如果 input_mode 配置为 INPUT_MODE_SUBLVDS/ INPUT_MODE_LVDS/ INPUT_MODE_HISPI, 则必须配置 lvds_attr

【解决方案差异】

解决方案	本结构体
SS928V100	支持

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.34 OT_MIPI_TX_IOC_MAGIC

【说明】

MIPI Tx ioctl 命令的幻数。

【定义】

```
#define OT_MIPI_TX_IOC_MAGIC    't'
```

【成员】

无

【注意事项】

无

【相关数据类型及接口】

无

1.5.35 LANE_MAX_NUM

【说明】

定义 MIPI TX 支持的最大 Lane 数。

【定义】

```
#define LANE_MAX_NUM 4
```

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.36 MIPI_TX_SET_DATA_SIZE

【说明】

定义 MIPI TX 长指令支持的最大数据长度。

【定义】

```
#define MIPI_TX_SET_DATA_SIZE 800
```

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.37 MIPI_TX_GET_DATA_SIZE

【说明】

定义 MIPI TX 读指令支持的最大数据长度。

【定义】

```
#define MIPI_TX_GET_DATA_SIZE 160
```

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.38 ATTRIBUTE

【说明】

定义编译时对齐字节数。

【定义】

```
#define OT_ALIGN_NUM      8
#define ATTRIBUTE         __attribute__((aligned(OT_ALIGN_NUM)))
```

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.39 out_mode_t

【说明】

MIPI TX 输出或外设操作模式。

【定义】

```
typedef enum {
    OUT_MODE_CSI           = 0x0,
    OUT_MODE_DSI_VIDEO     = 0x1,
    OUT_MODE_DSI_CMD       = 0x2,
    OUT_MODE_BUTT
} out_mode_t;
```

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.40 mipi_tx_work_mode_t

【说明】

MIPI Tx 工作模式

【定义】

```
typedef enum {  
    MIPI_TX_WORK_MODE_LP      = 0x0,  
    MIPI_TX_WORK_MODE_HS      = 0x1,  
    MIPI_TX_WORK_MODE_BUTT  
} mipi_tx_work_mode_t;
```

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.41 video_mode_t

【说明】

MIPI TX 视频模式或视频格式，或包序列格式。

【定义】

```
typedef enum {  
    BURST_MODE                = 0x0,  
    NON_BURST_MODE_SYNC_PULSES = 0x1,  
    NON_BURST_MODE_SYNC_EVENTS = 0x2,  
  
    VIDEO_DATA_MODE_BUTT  
} video_mode_t;
```

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.42 out_format_t

【说明】

MIPI TX 输出数据格式。

【定义】

```
typedef enum {  
    OUT_FORMAT_RGB_16BIT          = 0x0,  
    OUT_FORMAT_RGB_18BIT          = 0x1,  
    OUT_FORMAT_RGB_18BIT_LOOSELY = 0x2,  
    OUT_FORMAT_RGB_24BIT          = 0x3,  
    OUT_FORMAT_YUV420_12BIT       = 0x4,  
    OUT_FORMAT_YUV422_16BIT       = 0x5,  
  
    OUT_FORMAT_YUV420_8BIT_NORMAL = 0x6,  
    OUT_FORMAT_YUV420_8BIT_LEGACY = 0x7,  
    OUT_FORMAT_YUV422_8BIT        = 0x8,  
    OUT_FORMAT_RGB_888            = 0x9,  
    OUT_FORMAT_RAW_8BIT           = 0xa,  
    OUT_FORMAT_RAW_10BIT          = 0xb,  
    OUT_FORMAT_RAW_12BIT          = 0xc,  
    OUT_FORMAT_RAW_14BIT          = 0xd,  
    OUT_FORMAT_RAW_16BIT          = 0xe,  
    OUT_FORMAT_BUTT  
} out_format_t;
```

【注意事项】

- DSI 模式 (OUT_MODE_DSI_CMD, 或 OUT_MODE_DSI_VIDEO) 支持数据格式范围: [OUT_FORMAT_RGB_16BIT, OUT_FORMAT_YUV422_16BIT]。
- CSI 模式 (OUT_MODE_CSI) 支持数据格式范围:
[OUT_FORMAT_YUV420_8BIT_NORMAL, OUT_FORMAT_RAW_16BIT]。

【相关数据类型及接口】

无。

1.5.43 sync_info_t

【说明】

MIPI TX 设备同步信息。

【定义】

```
typedef struct {
    unsigned short  hsa_pixels;
    unsigned short  hbp_pixels;
    unsigned short  hact_pixels;
    unsigned short  hfp_pixels;

    unsigned short  vsa_lines;
    unsigned short  vbp_lines;
    unsigned short  vact_lines;
    unsigned short  vfp_lines;
} sync_info_t;
```

【成员】

成员名称	描述
hsa_pixels	水平同步脉冲区像素个数，取值范围：[1, 65535]。
hbp_pixels	水平后消隐区像素个数，取值范围：[1, 65535]。
hact_pixels	水平有效区像素个数，取值范围：[1, 4096]。
hfp_pixels	水平前消隐区像素个数，取值范围：[1, 65535]。
vsa_lines	垂直同步脉冲区行数，取值范围：[1, 256]。
vbp_lines	垂直后消隐区行数，取值范围：[1, 256]。
vact_lines	垂直有效区行数，取值范围：[100, 4096]。
vfp_lines	垂直前隐区行数，取值范围：[1, 256]。

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.44 combo_dev_cfg_t

【说明】

MIPI TX 设备属性。

【定义】

```
typedef struct {
    unsigned int    devno;                /* device number */
    short           lane_id[LANE_MAX_NUM]; /* lane_id: -1 - disable */
    out_mode_t      out_mode;             /* output mode:
CSI/DSI_VIDEO/DSI_CMD */
    video_mode_t    video_mode;
    out_format_t    out_format;
    sync_info_t     sync_info;
    unsigned int    phy_data_rate;        /* Mbps */
    unsigned int    pixel_clk;            /* KHz */
} combo_dev_cfg_t;
```

【成员】

成员名称	描述
devno	MIPI Tx 设备号，仅支持设备 0。
lane_id	lane 个数和模式配置，取决于 lane_id 中非-1 值的个数，仅四种模式可选，且 lane 个数、lane 号码、lane 顺序可调整： 配置方法参考本章 1.8.1 MIPI RX Lane id 如何配置 小节。 1 lane 模式：lane0 2 lane 模式：lane0, lane1 3 lane 模式：lane0, lane1, lane2 4 lane 模式：lane0, lane1, lane2, lane3 建议的配置方法： 1 lane 模式，配置成：lane_id[4] = {0, -1, -1, -1} 2 lane 模式，配置成：lane_id[4] = {0, 1, -1, -1} 3 lane 模式，配置成：lane_id[4] = {0, 1, 2, -1} 4 lane 模式，配置成：lane_id[4] = {0, 1, 2, 3}
out_mode	MIPI Tx 输出模式。
video_mode	MIPI Tx 视频模式。
out_format	MIPI Tx 输出格式。
sync_info	MIPI Tx 设备的同步信息。

成员名称	描述
phy_data_rate	MIPI Tx 输出速率，输出范围请参考芯片手册“MIPI Tx”章节，MIPI Tx 高速模式每个通道 (lane) 的速率范围的描述，单位是 Mbps。 取值范围：[80, 2500]
pixel_clk	像素时钟，单位为 KHz。 取值范围：[3333, 1250000]

【注意事项】

无。

【相关数据类型及接口】

无。

1.5.45 cmd_info_t

【说明】

发送给 MIPI TX 设备的命令信息。

【定义】

```
typedef struct {
    unsigned int      devno;
    mipi_tx_work_mode_t work_mode;
    unsigned char     lp_clk_en;
    unsigned short    data_type;
    unsigned short    cmd_size;
    const unsigned char ATTRIBUTE *cmd;
} cmd_info_t;
```

【成员】

成员名称	描述
devno	MIPI TX 设备号，仅支持设备 0。
work_mode	工作模式：低功耗模式，高速模式。
lp_clk_en	低功耗模式时钟开关。

成员名称	描述
data_type	命令数据类型。
cmd_size	短指令的数据，范围：[0, 0xFFFF]。 长指令的数据长度，范围：(0, MIPI_TX_SET_DATA_SIZE], 单位：字节。 注：cmd 置为 NULL：cmd_size 表示短指令的数据，低八位对应数据 1、高八位对应数据 2。 cmd 置为非 NULL：cmd_size 表示长指令的数据长度。
cmd	长指令数据指针。 • 设置短指令时，需置为 NULL； • 设置长指令时，需置为非 NULL。

【注意事项】

cmd 成员为 NULL 时，将不对 cmd_size 成员做取值范围检查。

【相关数据类型及接口】

无。

1.5.46 get_cmd_info_t

【说明】

发送给 MIPI TX 设备的命令信息。

【定义】

```
typedef struct {
    unsigned int      devno;
    mipi\_tx\_work\_mode\_t work_mode;
    unsigned char     lp_clk_en;
    unsigned short    data_type;
    unsigned short    data_param;
    unsigned short    get_data_size;
    unsigned char     ATTRIBUTE *get_data;
} get_cmd_info_t;
```

【成员】

成员名称	描述
devno	MIPI TX 设备号，仅支持设备 0。
work_mode	工作模式：低功耗模式，高速模式。
lp_clk_en	低功耗模式时钟开关。
data_type	命令数据类型。
data_param	数据参数，低八比特为第一个参数，高八比特为第二个参数、不用时填 0。
get_data_size	预期获取的数据字节数，范围：(0, MIPI_TX_GET_DATA_SIZE]。
get_data	获取到的数据存放地址指针，需要用户分配，不能设置为 NULL。

【注意事项】

无。

【相关数据类型及接口】

无。

1.6 MIPI TX 模块参数

1.6.1 g_smooth

g_smooth 模块参数用于实现平滑过渡，使用方法如下：

在 Linux 操作系统下，加载 MIPI TX 驱动的时候可以控制是否复位 MIPI TX 的逻辑。

- insmod ot_mipi_tx.ko smooth=1：加载 ko 时不进行逻辑复位。
- insmod ot_mipi_tx.ko smooth=0：加载 ko 时进行逻辑复位。

1.7 Proc 信息

1.7.1 MIPI_RX Proc 信息

MIPI_RX 正常工作状态下 proc 信息中宽高应该是稳定不变且和 sensor 输出时序的宽高匹配，并且 MIPI_RX 各种错误中断计数为 0。如果错误中断计数不为 0，请检查 MIPI_RX 相关属性是否配置正确。

【调试信息】

[MIPI_RX] Version: [SS928V100V1.0.0.0 B010 Release], Build Time[Jan 25 2022, 11:06:37]

```

----mipi lane divide mdoe-----
mode          lane divide
  0              8

----mipi dev attr-----
devno  work_mode  data_rate      data_type  wdr_mode  img_x  img_y  img_w
img_h
  0      mipi      X1              raw12     none    0      0    3840
2160

----mipi lane info-----
devno          lane_id
  0             0, 1, 2, 3, -1, -1, -1, -1

----mipi phy data info-----
phy_id      lane_id      phy_data      mipi_data      lvds_data      freq
  0          0, 1, 2, 3    0x00,0x00,0x00,0x00    0x00,0x00,0x00,0x00
0x00,0x00,0x00,0x00      n/a
  1          4, 5, 6, 7    0x00,0x00,0x00,0x00    0x00,0x00,0x00,0x00
0x00,0x00,0x00,0x00      n/a

----mipi detect info-----
devno vc    width  height
  0  0      1920  1080
  0  1         0    0
  0  2         0    0
  0  3         0    0

----lvds detect info-----
devno vc    width  height
  0  0      3840  2160
  0  1         0    0
  
```

```

0 2      0      0
0 3      0      0

----lvds lane detect info-----
devno  lane  width  height
0      2      960    2179
0      4      960    2179
0      5      960    2179
0      7      960    2179

----phy cil err int info-----
phy_id clk2_tmout clk_tmout lane0_tmout lane1_tmout lane2_tmout lane3_tmout clk2_esc
clk_esc lane0_esc lane1_esc lane2_esc lane3_esc
0      0      0      0      0      0      0      0      0
0      0      0      0      0
0      1      0      0      0      0      0      0      0
0      0      0      0      0

----mipi error int info1-----
devno  ecc2  vc0_crc  vc1_crc  vc2_crc  vc3_crc  vc0_ecc_corrc  vc1_ecc_corrc
vc2_ecc_corrc  vc3_ecc_corrc
0      0      0      0      0      0      0      0
0      0

----mipi error int info2-----
devno  vc0_frmnum  vc1_frmnum  vc2_frmnum  vc3_frmnum  vc0_frmmmatch
vc1_frmmmatch  vc2_frmmmatch  vc3_frmmmatch
0      0      0      0      0      0      0
0      0

----mipi error int info3-----
devno  data_fifo_rderr  cmd_fifo_rderr  vsync  cmd_fifo_wrerr  data_fifo_wrerr
0      0      0      0      0      0

----lvds error int info1-----
devno  vsync  cmd_rderr  cmd_wrerr  pop_err  stat_err
0      0      0      0      0      0

----lvds error int info2-----
devno  link0_wrerr  link1_wrerr  link2_wrerr  link0_rderr  link1_rderr  link2_rderr
0      0      0      0      0      0      0

----lvds error int info3-----

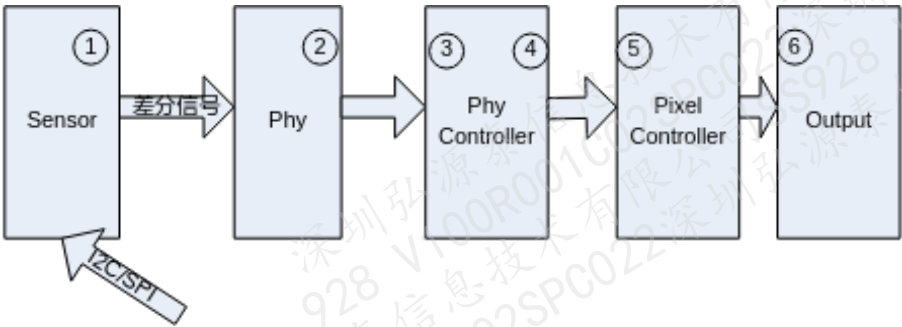
```

devno	lane0_err	lane1_err	lane2_err	lane3_err	lane4_err	lane5_err	lane6_err	lane7_err
0	0	0	0	0	0	0	0	0
-----align error int info-----								
devno	fifo_fullerr	lane0_err	lane1_err	lane2_err	lane3_err	lane4_err	lane5_err	lane6_err
lane6_err	lane7_err							
0	0	0	0	0	0	0	0	0
0	0	0						

【调试信息分析】

- MIPI_Rx 通过 Phy 接收 sensor 的差分数据，Phy Controller 检测到同步头后，将每条 lane 上的数据对齐；
- Pixel Controller 解析同步信息并按照 raw data 的位宽将 lane 上面的数据合并为 Pixel 数据；Output 模式将 Pixel 数据发送给后级模块。
- Phy PhyController PixelController 由 sensor 的 pixel clk 提供时钟，output 模块的时钟为称为随路时钟，与后级模块的工作时钟相同。MIPI_Rx 的 crop 功能在 Pixel Controller 的末端实现，所以 Crop 后可以降低需要的随路时钟。

图1-3 MIPI 数据流



【参数说明】

参数		描述
mipi lane divide mdoe	mode	MIPI Rx 的 lane 分布模式
	lane divide	MIPI_Rx 详细的 LANE 分布。
mipi dev	devno	MIPI 设备号

参数		描述
attr	work_mode	MIPI 设备工作模式：LVDS/MIPI 等模式
	data_rate	MIPI Rx 的速率。
	data_type	数据类型：RAW8 / RAW10/ RAW12/ RAW14/ RAW16 bit 等类型
	wdr_mode	WDR 模式： • None：非 WDR 模式 • 2To1：2 合 1 WDR • 3To1：3 合 1 WDR • 4To1：4 合 1 WDR • DOL2To1：DOL2 合 1WDR • DOL3To1：DOL3 合 1WDR • DOL4To1：DOL4 合 1WDR
	img_x	Crop 图像起始 X
	img_y	Crop 图像起始 Y
	img_w	Crop 图像宽度
	img_h	Crop 图像高度
mipi lane info	devno	MIPI 设备号
	lane_id	Lane ID
mipi phy data info	phy_id	PHY ID 序号
	lane_id	对应的 Lane Id
	phy_data	PHY 对应的 4 条 Lane 接收到的实时数据
	mipi_data	PHY 对应的 4 条 Lane 检测到 MIPI 帧的数据包头数据
	lvds_data	PHY 对应的 4 条 Lane 检测到 LVDS 帧同步信号后的实时数据
	freq	HS 时钟频率测量值

参数		描述
mipi detect info (仅 MIPI 模式下可见)	devno	MIPI_Rx 设备号
	vc	Virtual Channel
	width	MIPI 控制器检测到的图像总宽度
	height	MIPI 控制器检测到的图像总高度
lvds detect info (仅 LVDS 模式下可见)	devno	MIPI_Rx 设备号
	vc	Virtual Channel
	width	MIPI 控制器检测到的图像总宽度
	height	MIPI 控制器检测到的图像总高度
lvds lane detect info (仅 LVDS 模式下可见)	devno	MIPI_Rx 设备号
	lane	Lane ID
	width	该 lane 上检测到的宽度。
	height	该 lane 上检测到的高度。
phy cil err int info	phy_id	PHY ID
	clk2_tmout	Clock2 Lane 从 LP 切换到 HS 超时
	clk_tmout	Clock1 Lane 从 LP 切换到 HS 超时
	lane0_tmout	DataLane 0 从 LP 切换到 HS 超时
	lane1_tmout	Data Lane 1 从 LP 切换到 HS 超时
	lane2_tmout	Data Lane 2 从 LP 切换到 HS 超时
	lane3_tmout	Data Lane 3 从 LP 切换到 HS 超时
	clk2_esc	Clock2 Lane 切换到 escape 模式超时
	clk_esc	Clock Lane 切换到 escape 模式超时
	lane0_esc	Data Lane 0 切换到 escape 模式超时
	lane1_esc	Data Lane 1 切换到 escape 模式超时
	lane2_esc	Data Lane 2 切换到 escape 模式超时

参数		描述
	lane3_esc	Data Lane 3 切换到 escape 模式超时
mipi error int info1 (仅 MIPI 模式下可 见)	devno	MIPI_Rx 设备号。
	ecc2	Header 至少 2 个错误, ECC 无法纠错
	vc0_crc	VC0 通道数据的 CRC 错误计数。
	vc1_crc	VC1 通道数据的 CRC 错误计数。
	vc2_crc	VC2 通道数据的 CRC 错误计数。
	vc3_crc	VC3 通道数据的 CRC 错误计数。
	vc0_ecc_corrct	VC0 通道 Header 的 ECC 已纠正的错误计数。
	vc1_ecc_corrct	VC1 通道 Header 的 ECC 已纠正的错误计数。
	vc2_ecc_corrct	VC2 通道 Header 的 ECC 已纠正的错误计数。
	vc3_ecc_corrct	VC3 通道 Header 的 ECC 已纠正的错误计数。
mipi error int info2 (仅 MIPI 模式下可 见)	devno	MIPI_Rx 设备号。
	vc0_frmnum	VC0 的帧序错误计数
	vc1_frmnum	VC1 的帧序错误计数
	vc2_frmnum	VC2 的帧序错误计数
	vc3_frmnum	VC3 的帧序错误计数
	vc0_frmmmatch	VC0 通道的帧起始与帧结束短包不匹配计数
	vc1_frmmmatch	VC1 通道的帧起始与帧结束短包不匹配计数
	vc2_frmmmatch	VC2 通道的帧起始与帧结束短包不匹配计数
	vc3_frmmmatch	VC3 通道的帧起始与帧结束短包不匹配计数
mipi error int info3 (仅 MIPI 模式下可 见)	devno	MIPI_Rx 设备号。
	data_fifo_rderr	MIPI CTRL 读数据 FIFO 中断计数
	cmd_fifo_rderr	MIPI CTRL 读命令 FIFO 中断计数
	vsync	MIPI CTR vsync 中断计数

参数		描述
	cmd_fifo_wrerr	MIPI CTRL 写命令 FIFO 中断计数
	data_fifo_wrerr	MIPI CTRL 写数据 FIFO 中断计数
lvds error int info1 (仅 LVDS 模式下可 见)	devno	MIPI 设备号
	vsync	lvds vsync 中断计数
	cmd_rderr	lvds CMD_FIFO 读错误中断计数
	cmd_wrerr	lvds CMD_FIFO 写错误中断计数
	pop_err	lvds 读取 line_buf 错误中断计数
	stat_err	lvds 各 lane 同步错误中断计数
lvds error int info2 (仅 LVDS 模式下可 见)	devno	MIPI 设备号
	link0_wrerr	link0 写 FIFO 错误中断计数
	link1_wrerr	link1 写 FIFO 错误中断计数
	link2_wrerr	link2 写 FIFO 错误中断计数
	link0_rderr	link0 读 FIFO 错误中断计数
	link1_rderr	link1 读 FIFO 错误中断计数
	link2_rderr	link2 读 FIFO 错误中断计数
lvds error int info3 (仅 LVDS 模式下可 见)	devno	MIPI 设备号
	lane0_err	Lane0 同步错误中断计数
	lane1_err	Lane1 同步错误中断计数
	lane2_err	Lane2 同步错误中断计数
	lane3_err	Lane3 同步错误中断计数
	lane4_err	Lane4 同步错误中断计数
	lane5_err	Lane5 同步错误中断计数
	lane6_err	Lane6 同步错误中断计数
	lane7_err	Lane7 同步错误中断计数

参数		描述
align error int info	devno	MIPI 设备号
	fifo_fullerr	FIFO 溢出
	lane0_err	Lane0 FIFO 溢出
	lane1_err	Lane1 FIFO 溢出
	lane2_err	Lane2 FIFO 溢出
	lane3_err	Lane3 FIFO 溢出
	lane4_err	Lane4 FIFO 溢出
	lane5_err	Lane5 FIFO 溢出
	lane6_err	Lane6 FIFO 溢出
	lane7_err	Lane7 FIFO 溢出

1.7.2 MIPI_TX Proc 信息

MIPI_TX 的 proc 信息主要有：MIPI_TX 设备模块参数、MIPI_TX 设备配置信息、MIPI_TX 时序配置信息、MIPI_TX 设备状态信息。

【调试信息】

[MIPI_TX] Version: [SS928V100V1.0.0.0 B010 Release], Build Time[Jan 25 2022, 11:07:02]

-----mipi_tx module config-----

smooth
0

----- mipi_tx dev config-----

devno	lane0	lane1	lane2	lane3	output_mode	phy_data_rate	pixel_clk(KHz)
0	0	1	2	3	1	945	148500
0	2						

----- mipi_tx sync config-----

hact	hbp	hfp	hsa	vact	vbp
1080	20	130	8	1920	26

```

16          10

----- mipi_tx dev status-----
width  height horiall vertall hbp hsa vsa
1080    1920    1237    1972    20   8   10

```

【调试信息分析】

MIPI_TX 模块参数、MIPI_TX 设备配置信息、MIPI_TX 时序配置信息等为设备启动前通过接口配置的信息；MIPI_TX 设备状态信息是设备运行时检测到的部分时序信息：有效宽高、水平总宽度、垂直总高度。

【参数说明】

参数		描述
mipi_tx module config	smooth	平滑过渡标记： 0：关闭。 1：开启。
	devno	设备号。
mipi_tx dev config	lane0 lane1 lane2 lane3	>=0：lane 号码。 -1：禁用。
	output_mode	0: csi mode 1: dsi video mode 2: dsi command mode
	phy_data_rate	Phy 数据率。
	pixel_clk(KHz)	像素时钟(单位 KHz)。
	video_mode	0: BURST_MODE 1: NON_BURST_MODE_SYNC_PULSES 2: NON_BURST_MODE_SYNC_EVENTS
	output_fmt	0: OUT_FORMAT_RGB_16BIT 1: OUT_FORMAT_RGB_18BIT 2: OUT_FORMAT_RGB_18BIT_LOOSELY 3: OUT_FORMAT_RGB_24BIT 4: OUT_FORMAT_YUV420_12BIT 5: OUT_FORMAT_YUV422_16BIT 6: OUT_FORMAT_YUV420_8BIT_NORMAL

参数		描述
		7: OUT_FORMAT_YUV420_8BIT_LEGACY 8: OUT_FORMAT_YUV422_8BIT 9: OUT_FORMAT_RGB_888 10: OUT_FORMAT_RAW_8BIT 11: OUT_FORMAT_RAW_10BIT 12: OUT_FORMAT_RAW_12BIT 13: OUT_FORMAT_RAW_14BIT 14: OUT_FORMAT_RAW_16BIT
mipi_tx sync config	hact	水平有效区。
	hbp	水平后消隐区。
	hfp	水平前消隐区。
	hsa	水平同步脉冲区。
	vact	垂直有效区。
	vbp	垂直后消隐区。
	vfp	垂直前消隐区。
	vsa	垂直同步脉冲区。
mipi_tx dev status	width	检测到的宽。
	height	检测到的高。
	horiall	水平总宽。
	vertall	垂直总高。
	hbp	水平前肩。
	hsa	水平同步脉冲个数。
	vsa	垂直同步脉冲个数。

1.8 FAQ

MIPI 具体规格请参考芯片手册。

1.8.1 MIPI RX Lane id 如何配置

Lane id 的配置对应 `mipi_dev_attr_t` 中的 `short lane_id[MIPI_LANE_NUM]`，其中 `lane_id` 数组的索引号表示的是 SENSOR 的 LANE ID，`lane_id` 数组的值表示的是 MIPI 的 LANE ID。

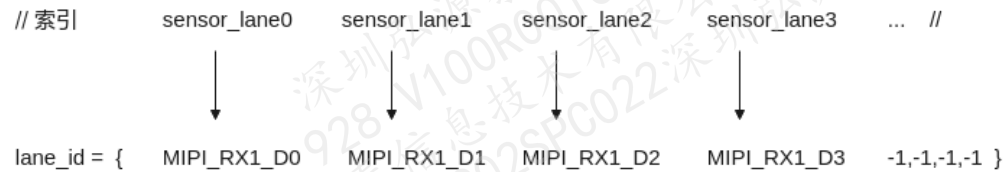
对接 sensor 时，未使用的 lane 将其对应的 `lane_id` 配置为-1。配置 `lane_id` 还可以调整数据通道顺序，根据硬件单板与实际 sensor 输出通道的对应关系调整 `lane_id` 的配置。

下面举例进行说明，例如 MIPI 与 SENSOR 的引脚硬件连接如表 1-6 所示。

表1-6 SENSOR 与 MIPI_Rx 管脚关系

MIPI Lane 管脚	SENSOR 管脚
MIPI_RX1_D0	Lane 0
MIPI_RX1_D1	Lane 1
MIPI_RX1_D2	Lane 2
MIPI_RX1_D3	Lane 3

MIPI 的最大 Lane 数为 8，我们认为 SENSOR 的 Lane 数目最多 8 个，由于 sensor 实际只有 4 个 Lane，只输出数据到 MIPI 的 4 个 Lane，需要将 SENSOR 未连接的或者不存在的 Lane 的 `lane_id` 配置为-1，所以 `lane_id` 配置如下：



1.8.2 MIPI TX Lane id 如何配置

Lane id 的配置对应 `combo_dev_cfg_t` 中的 `short lane_id[LANE_MAX_NUM]`，其中 `lane_id` 数组的索引号即数组下标表示的是 MIPI TX 对端设备的 LANE ID，`lane_id` 数组的值表示的是 MIPI TX 的 LANE ID。

对接屏幕时，未使用的 lane 将其对应的 `lane_id` 配置为-1。配置 `lane_id` 还可以调整数据通道顺序，根据硬件单板与实际屏幕输出通道的对应关系调整 `lane_id` 的配置。

下面举例进行说明，例如 MIPI TX 与屏幕的引脚硬件连接如表 1-7 所示。

表1-7 4lane 屏幕与 MIPI_TX 管脚关系

MIPI TX Lane 管脚	屏幕管脚
MIPI_TX_D2	dev_lane0
MIPI_TX_D3	dev_lane1
MIPI_TX_D0	dev_lane2
MIPI_TX_D1	dev_lane3

MIPI TX 的最大 Lane 数为 4，屏幕实际可能有 4、2 个 Lane，只使用到 MIPI TX 的 4 个 Lane 或 2 个 lane，需要将屏幕端未连接的或者不存在的 Lane 的 lane_id 配置为 -1，所以 4lane 屏幕时，lane_id 配置如下：

```
// 索引      dev_lane0      dev_lane1      dev_lane2      dev_lane3      //
              ↑              ↑              ↑              ↑
lane_id = {   MIPI_TX_D2   MIPI_TX_D3   MIPI_TX_D0   MIPI_TX_D1   }
```

表1-8 2lane 屏幕与 MIPI_TX 管脚关系

MIPI TX Lane 管脚	屏幕管脚
MIPI_TX_D1	dev_lane0
MIPI_TX_D0	dev_lane1
MIPI_TX_D2	未连接
MIPI_TX_D3	未连接

2lane 屏幕时，lane_id 配置如下：

```
// 索引      dev_lane0      dev_lane1      dev_lane2      dev_lane3      //
              ↑              ↑
lane_id = {   MIPI_TX_D1   MIPI_TX_D0   -1           -1   }
```

1.8.3 MIPI 频率说明

1.8.3.1 MIPI Lane 频率与 VI 频率关系

使用 MIPI 多个 Lanes 进行数据传输，MIPI Lane 的传输频率与 VI 处理频率如何对应，每一 Lane 可传输的最高速率如何计算？

- MIPI_Rx 使用多 Lane 接收数据，会转成内部时序，送给 VI 模块进行处理，多 Lane 传输的数据总量不变，有这样的计算公式：
$$VI_Freq * Pix_Width = Lane_Num * MIPI_Freq$$
- 其中，VI_Freq 为 VI 的工作时钟，Pix_Width 为像素位宽，Lane_Num 为传输 lane 个数，MIPI_Freq 为一个 lane 能接收的最大频率。
- 下面以 VI 工作频率为 250M，MIPI 数据为 RAW 12, 4Lane 传输为例进行说明：
$$MIPI_Freq = (250 * 12) / 4 = 750$$

即每个 Lane 最高频率为 750MHz